Operating Systems and System Programming

Skill-2

Task-1:

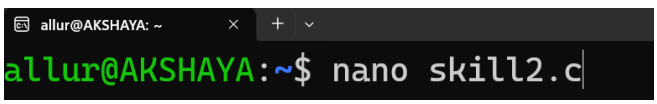
Aim:

To develop and test an interactive command-line loop that displays a prompt, reads user input, handles exit conditions, captures keyboard input including backspace and Enter keys, manages the input buffer, and supports multi-character commands.

Algorithm

1. Start the program.
2. Display the command prompt.
3. Read characters entered by the user.
4. Store the characters in an input buffer.
5. If a normal character is entered, add it to the buffer.
6. If Backspace is pressed, remove the last character from the buffer.
7. If Enter is pressed, terminate the current input and process the command.
8. Check whether the entered command is an exit command.
9. If it is an exit command, terminate the loop and end the program.
10. Otherwise, process and display the entered command.
11. Return to the main loop and display the prompt again.
12. Test the loop with single-character and multi-character commands.
13. Stop the program.

Step-1:



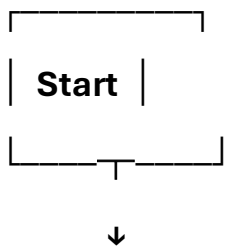
```
allur@AKSHAYA: ~
 GNU nano 8.7.1                                skill2.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[50];

    while (1)
    {
        printf("myshell> ");
        scanf("%s", input);

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J
```

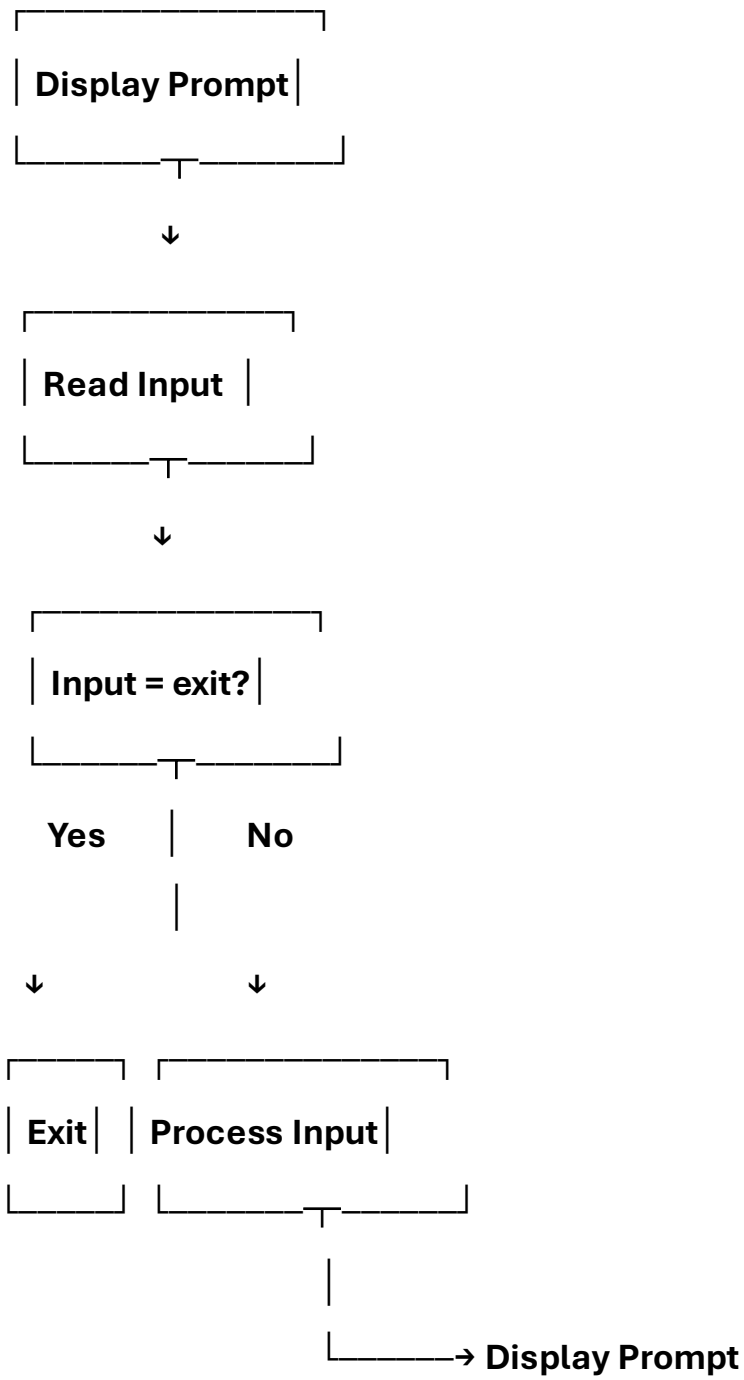
```
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Step-2:Compile and Run

```
allur@AKSHAYA:~$ gcc skill2.c
allur@AKSHAYA:~$ ./a.out
myshell> hello
You entered: hello
myshell> ls
You entered: ls
myshell> exit
allur@AKSHAYA:~$
```

Step-3: Control Flow Diagram




Display Prompt
Read Input
Input = exit?
Yes
No
Exit
Process Input
Display Prompt

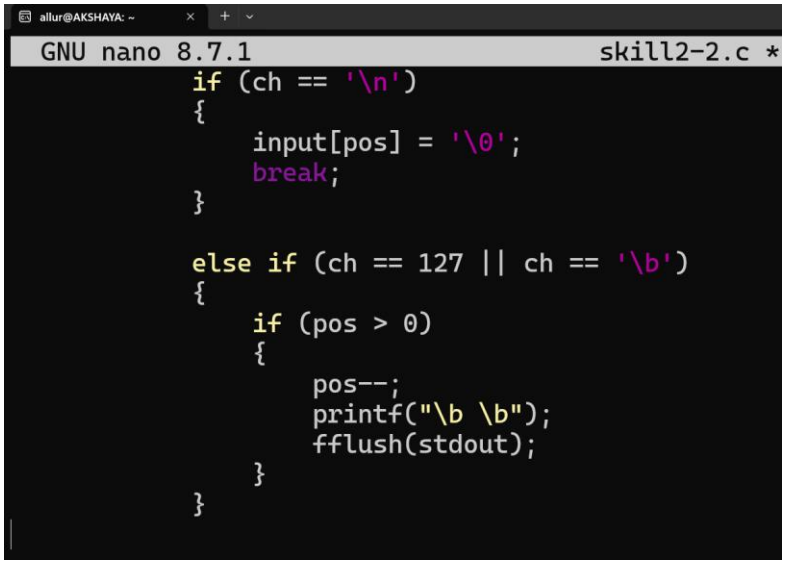
Task-2:

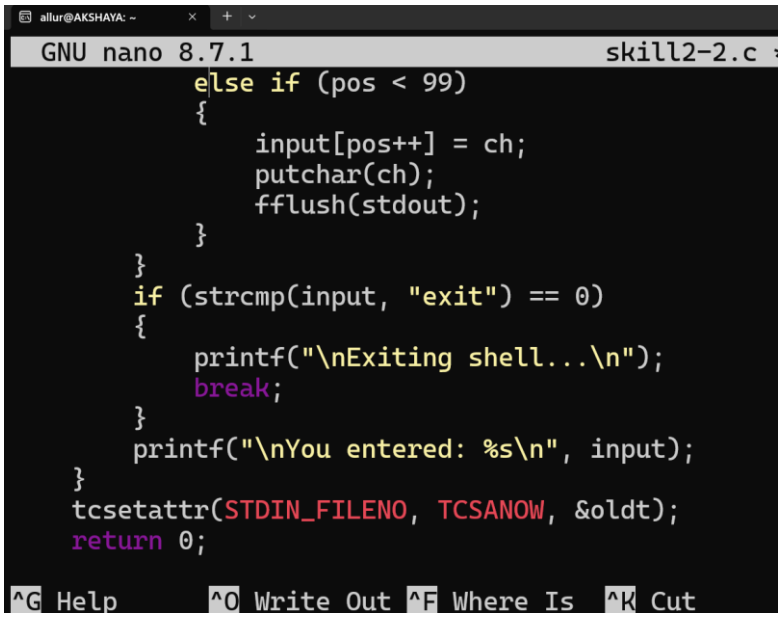
Step-1:

```
allur@AKSHAYA:~$ nano skill2-2.c
```

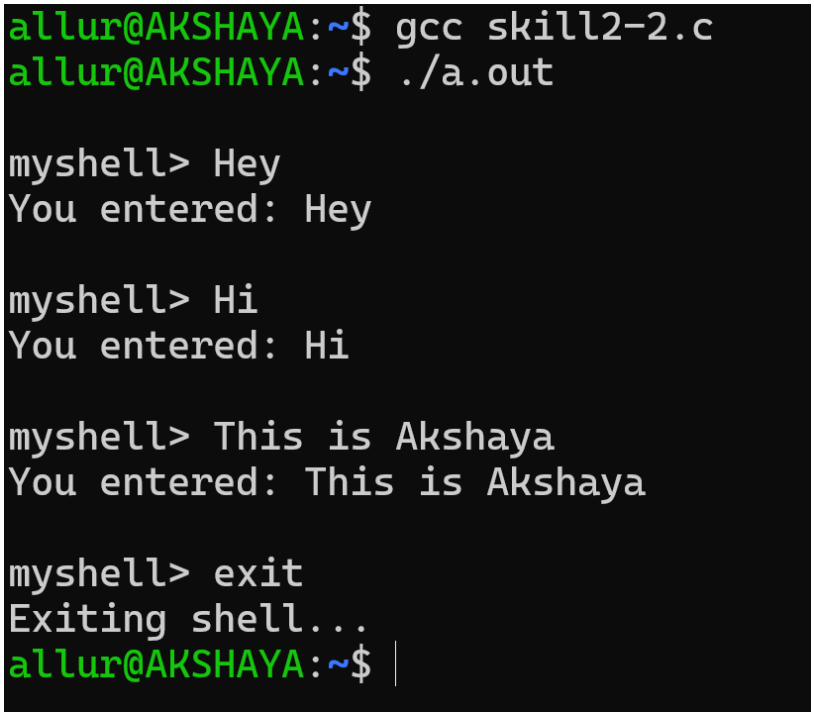
```
allur@AKSHAYA: ~
  GNU nano 8.7.1                                skill2-2.c *
#include <stdio.h>
#include <termios.h>
#include <unistd.h>
#include <string.h>

int main()
{
    char input[100];
    char ch;
    int pos;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;

^G Help       ^O Write Out ^F Where Is  ^K Cut
^X Exit       ^R Read File ^\ Replace   ^U Paste
```

```
allur@AKSHAYA: ~
  GNU nano 8.7.1                                skill2-2.c *
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        printf("\nmyshell> ");
        fflush(stdout);

        pos = 0;

        while (1)
        {
            ch = getchar();

            if (ch == '\n')
            {

^G Help       ^O Write Out ^F Where Is  ^K Cut
```


allur@AKSHAYA: ~
GNU nano 8.7.1 skill2-2.c *
if (ch == '\n')
{
input[pos] = '\0';
break;
}
else if (ch == 127 || ch == '\b')
{
if (pos > 0)
{
pos--;
printf("\b \b");
fflush(stdout);
}
}


allur@AKSHAYA: ~
GNU nano 8.7.1 skill2-2.c
else if (pos < 99)
{
input[pos++] = ch;
putchar(ch);
fflush(stdout);
}
}
if (strcmp(input, "exit") == 0)
{
printf("\nExiting shell...\n");
break;
}
printf("\nYou entered: %s\n", input);
}
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
return 0;
^G Help ^O Write Out ^F Where Is ^K Cut

Step-2:



Result

The interactive command-line loop was successfully designed and tested. It correctly displays the prompt, accepts keyboard input, handles Backspace and Enter, manages the input buffer, supports multi-character commands, and terminates when an exit condition is received.